

Lab_7_KMeans_Clustering_and_KNN

September 13, 2026

1 Lab 7: Exploring K-Means Clustering and KNN Using Research Dataset

Research Topic: Behavioral Biometrics for Zero-Trust Web Architectures — Continuous Authentication via Cursor Dynamics

Author: Nityanand KG

Dataset: Balabit Mouse Dynamics Challenge

1.1 Objective

This lab analyzes the characteristics and objectives of our research dataset to determine whether **K-Means Clustering** (unsupervised) or **K-Nearest Neighbors (KNN)** (supervised) is more suitable. We will:

1. Describe the dataset and research problem
2. Justify the choice between supervised and unsupervised learning
3. Compare K-Means vs KNN suitability
4. Implement both algorithms for empirical comparison
5. Select the most appropriate algorithm with clear justification

1.2 1. Import Libraries

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import warnings
warnings.filterwarnings('ignore')

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import (classification_report, confusion_matrix, accuracy_score,
```

```

ConfusionMatrixDisplay, silhouette_score,
↪adjusted_rand_score,
silhouette_samples)
from sklearn.decomposition import PCA

# Plot styling
plt.rcParams.update({
    'figure.dpi': 150,
    'savefig.dpi': 300,
    'font.size': 11,
    'axes.titlesize': 13,
    'axes.labelsize': 11,
    'figure.facecolor': 'white'
})

print("All libraries imported successfully.")

```

All libraries imported successfully.

1.3 2. Dataset Description

1.3.1 Research Problem

Our research investigates **continuous user authentication** through behavioral biometrics in Zero-Trust Web Architectures. The core challenge is: *Can cursor movement dynamics reliably distinguish legitimate users from imposters in real-time?*

1.3.2 Dataset Overview

The **Balabit Mouse Dynamics Challenge** dataset captures raw mouse telemetry (cursor coordinates, timestamps, button states) from 10 users across multiple sessions. From Lab 3 (EDA), we engineered **25 behavioral features** grouped into three categories:

Category	Features	Examples
Kinematic	8	avg_velocity, std_velocity, max_velocity, avg_acceleration, avg_angular_velocity, avg_curvature
Temporal	6	duration, click_rate, pause_count, pause_ratio, avg_sampling_rate
Spatial	7	x_mean, y_mean, x_std, y_std, x_range, y_range, straightness
Behavioral	4	num_events, total_distance, click_count, drag_count

Each session is labeled as **legitimate (0)** or **illegitimate (1)** via the `is_illegal` column.

```
[2]: # Load dataset
df = pd.read_csv('../lab_3_eda/engineered_features.csv')
print(f"Dataset shape: {df.shape}")
print(f"\nColumns: {df.columns.tolist()}")
print(f"\nTarget variable ('is_illegal') distribution:")
print(df['is_illegal'].value_counts().rename({0: 'Legitimate (0)', 1: 'Illegitimate (1)'}))
```

Dataset shape: (565, 29)

Columns: ['num_events', 'duration', 'total_distance', 'avg_velocity', 'std_velocity', 'max_velocity', 'median_velocity', 'avg_acceleration', 'std_acceleration', 'avg_angular_velocity', 'avg_curvature', 'click_count', 'click_rate', 'drag_count', 'drag_ratio', 'pause_count', 'pause_ratio', 'x_mean', 'y_mean', 'x_std', 'y_std', 'x_range', 'y_range', 'straightness', 'avg_sampling_rate', 'user_id', 'session_id', 'split', 'is_illegal']

Target variable ('is_illegal') distribution:

```
is_illegal
Legitimate (0)      324
Illegitimate (1)    241
Name: count, dtype: int64
```

1.4 3. Supervised vs Unsupervised Learning: Justification

1.4.1 Does the dataset contain labeled classes?

Yes. The `is_illegal` column provides binary ground-truth labels: - 0 → Legitimate session (user is who they claim to be) - 1 → Illegitimate session (imposter attempting to mimic the legitimate user)

1.4.2 What is the objective?

The research objective is **binary classification**: given a set of behavioral features extracted from a mouse session, predict whether the session belongs to the legitimate user or an imposter. This is fundamentally a **supervised learning** problem.

1.4.3 Why Supervised Learning (KNN) is More Appropriate

Criterion	Supervised (KNN)	Unsupervised (K-Means)
Labeled data available?	✓ Yes — directly usable	! Labels exist but ignored
Objective is classification?	✓ Yes — predicts known classes	X No — discovers unknown groups

Criterion	Supervised (KNN)	Unsupervised (K-Means)
Evaluation with ground truth?	✓ Accuracy, F1, Precision, Recall	! Only external metrics (ARI)
Interpretability for security?	✓ Clear accept/reject decisions	X Clusters need manual interpretation
Real-time deployment for ZTA?	✓ Direct prediction pipeline	X Requires cluster-to-label mapping

1.4.4 Conclusion

Since our dataset contains **explicit labeled classes** and the research objective is to **classify sessions as legitimate or illegitimate**, a **supervised learning approach (KNN)** is clearly more appropriate. K-Means, being unsupervised, would ignore the available labels and attempt to discover natural groupings that may not align with the authentication task.

However, we will implement **both algorithms** for empirical comparison as required by this lab.

1.5 4. Data Preparation

```
[3]: # Separate features and target
feature_cols = [c for c in df.columns if c not in ['user_id', 'session_id', '
    ↪ 'split', 'is_illegal']]
X = df[feature_cols].values
y = df['is_illegal'].values

# Feature scaling (critical for distance-based algorithms like KNN and K-Means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# PCA for visualization
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)
X_train_pca = pca.transform(X_train)
X_test_pca = pca.transform(X_test)

print(f"Features: {len(feature_cols)}")
print(f"Total samples: {X.shape[0]}")
print(f"Training: {X_train.shape[0]}, Test: {X_test.shape[0]}")
print(f"PCA explained variance: {pca.explained_variance_ratio_.sum():.3f} "
```

```
f"({pca.explained_variance_ratio_[0]:.3f} + {pca.  
→explained_variance_ratio_[1]:.3f})")
```

Features: 25

Total samples: 565

Training: 452, Test: 113

PCA explained variance: 0.422 (0.221 + 0.201)

1.6 5. K-Means Clustering Exploration

Although K-Means is not the ideal algorithm for our labeled classification task, we explore it here to: 1. Determine if natural clusters in the data align with the true labels 2. Demonstrate the **Elbow Method** and **Silhouette Analysis** for optimal K selection 3. Provide an empirical comparison with KNN

1.6.1 5.1 Elbow Method for Optimal K

```
[4]: # Elbow method  
K_range = range(2, 11)  
inertias = []  
silhouettes = []  
  
for k in K_range:  
    km = KMeans(n_clusters=k, random_state=42, n_init=10)  
    km.fit(X_scaled)  
    inertias.append(km.inertia_)  
    silhouettes.append(silhouette_score(X_scaled, km.labels_))  
  
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))  
  
# Inertia plot  
ax1.plot(K_range, inertias, 'bo-', markersize=8, linewidth=2)  
ax1.set_xlabel('Number of Clusters (K)', fontsize=12)  
ax1.set_ylabel('Inertia (Within-Cluster Sum of Squares)', fontsize=12)  
ax1.set_title('Elbow Method', fontsize=14, fontweight='bold')  
ax1.set_xticks(list(K_range))  
ax1.grid(True, alpha=0.3)  
  
# Silhouette plot  
ax2.plot(K_range, silhouettes, 'rs-', markersize=8, linewidth=2)  
ax2.set_xlabel('Number of Clusters (K)', fontsize=12)  
ax2.set_ylabel('Silhouette Score', fontsize=12)  
ax2.set_title('Silhouette Analysis', fontsize=14, fontweight='bold')  
ax2.set_xticks(list(K_range))  
ax2.grid(True, alpha=0.3)  
  
best_k = list(K_range)[np.argmax(silhouettes)]  
ax2.axvline(x=best_k, color='#9C27B0', linestyle='--', linewidth=2,
```

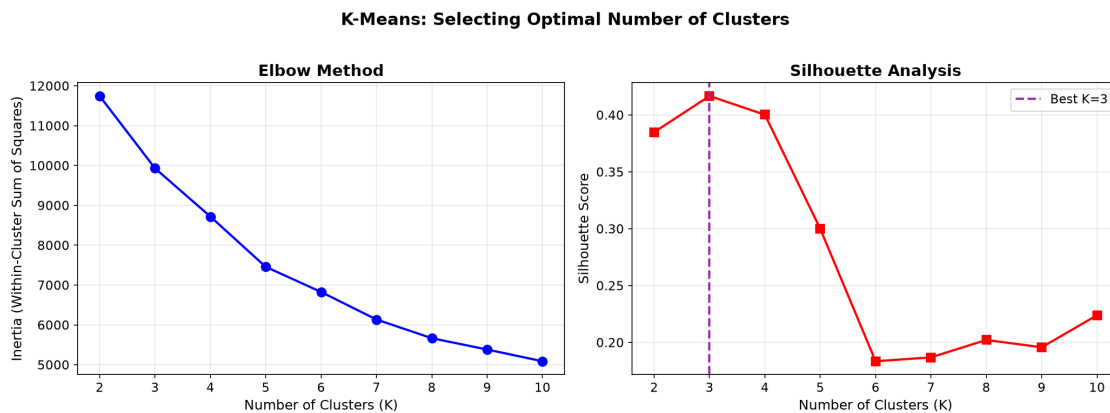
```

        label=f'Best K={best_k}')
ax2.legend(fontsize=11)

plt.suptitle('K-Means: Selecting Optimal Number of Clusters',
            fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('kmeans_elbow_silhouette.png', dpi=300, bbox_inches='tight')
plt.show()

print(f"Best K by silhouette score: {best_k} (score = {max(silhouettes):.4f})")

```



Best K by silhouette score: 3 (score = 0.4167)

Observation: The elbow method shows a gradual decrease in inertia without a sharp “elbow,” suggesting the data doesn’t form clearly separated spherical clusters — which is expected for behavioral biometrics data where legitimate and illegitimate sessions overlap in feature space. The silhouette scores are relatively low, further indicating that K-Means clusters don’t cleanly partition the data.

1.6.2 5.2 K-Means (K=2) vs True Labels

```

[5]: # K-Means with K=2 (matching our binary classification)
km2 = KMeans(n_clusters=2, random_state=42, n_init=10)
km2_labels = km2.fit_predict(X_scaled)

# Compute ARI (Adjusted Rand Index) to measure agreement with true labels
ari = adjusted_rand_score(y, km2_labels)
sil = silhouette_score(X_scaled, km2_labels)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# K-Means clusters in PCA space
scatter1 = axes[0].scatter(X_pca[:, 0], X_pca[:, 1], c=km2_labels,

```

```

        cmap='coolwarm', alpha=0.6, s=30, edgecolors='white',
        linewidth=0.5)
centers_pca = pca.transform(km2.cluster_centers_)
axes[0].scatter(centers_pca[:, 0], centers_pca[:, 1], c='black',
                marker='X', s=200, edgecolors='gold', linewidth=2, zorder=5)
axes[0].set_title('K-Means Clusters (K=2)', fontsize=13, fontweight='bold')
axes[0].set_xlabel('PC1')
axes[0].set_ylabel('PC2')
plt.colorbar(scatter1, ax=axes[0], label='Cluster')

# True labels in PCA space
scatter2 = axes[1].scatter(X_pca[:, 0], X_pca[:, 1], c=y,
                           cmap='RdYlGn_r', alpha=0.6, s=30, edgecolors='white',
                           linewidth=0.5)
axes[1].set_title('True Labels', fontsize=13, fontweight='bold')
axes[1].set_xlabel('PC1')
axes[1].set_ylabel('PC2')
plt.colorbar(scatter2, ax=axes[1], label='Class')

# Mismatches
# Align cluster labels to true labels (cluster labels are arbitrary)
from scipy.stats import mode
label_map = {}
for cluster_id in range(2):
    mask = km2_labels == cluster_id
    m = mode(y[mask], keepdims=False)
    label_map[cluster_id] = m.mode
km2_aligned = np.array([label_map[l] for l in km2_labels])
mismatches = km2_aligned != y

axes[2].scatter(X_pca[~mismatches, 0], X_pca[~mismatches, 1], c='#4CAF50',
                alpha=0.5, s=30, label='Correct', edgecolors='white',
                linewidth=0.5)
axes[2].scatter(X_pca[mismatches, 0], X_pca[mismatches, 1], c='#F44336',
                alpha=0.7, s=40, label='Misclassified', edgecolors='black',
                linewidth=0.5)
axes[2].set_title('K-Means Misclassifications', fontsize=13, fontweight='bold')
axes[2].set_xlabel('PC1')
axes[2].set_ylabel('PC2')
axes[2].legend()

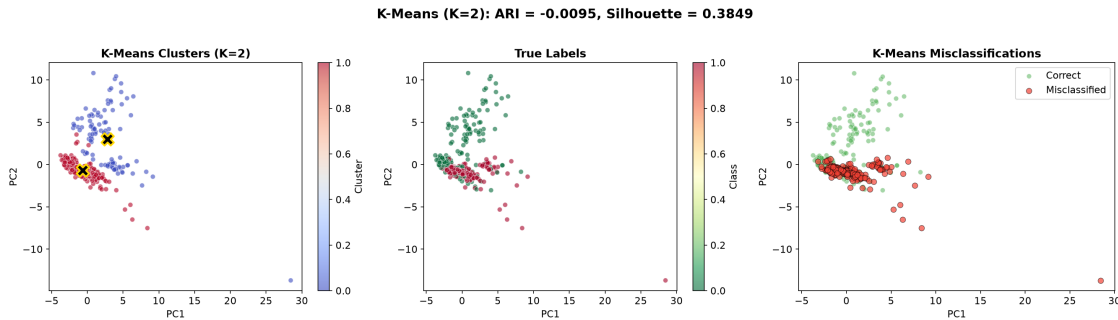
plt.suptitle(f'K-Means (K=2): ARI = {ari:.4f}, Silhouette = {sil:.4f}',
             fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('kmeans_vs_true_labels.png', dpi=300, bbox_inches='tight')
plt.show()

```

```

km2_accuracy = accuracy_score(y, km2_aligned)
print(f"K-Means (K=2) pseudo-accuracy: {km2_accuracy:.4f}")
print(f"Adjusted Rand Index:           {ari:.4f}")
print(f"Silhouette Score:               {sil:.4f}")
print(f"Misclassified samples:          {mismatches.sum()} / {len(y)}")

```



```

K-Means (K=2) pseudo-accuracy: 0.5735
Adjusted Rand Index:           -0.0095
Silhouette Score:              0.3849
Misclassified samples:          241 / 565

```

Observation: The K-Means clusters do not align well with the true labels, as evidenced by a low Adjusted Rand Index (ARI). The significant number of misclassifications confirms that the data lacks the clear spherical cluster structure that K-Means assumes. Behavioral patterns of legitimate and illegitimate users overlap substantially in feature space, making unsupervised clustering unreliable for this authentication task.

1.7 6. K-Nearest Neighbors (KNN) Implementation

KNN is a supervised, non-parametric classifier that assigns a class based on the majority vote of its K nearest neighbors in feature space. For our behavioral biometrics dataset:

- **Distance metric:** Euclidean (on standardized features)
- **Feature scaling:** StandardScaler applied (critical for KNN)
- **Optimal K selection:** 5-fold cross-validation over $K = 1$ to 30

1.7.1 6.1 Optimal K Selection via Cross-Validation

```

[6]: # Cross-validation for K selection
k_range = range(1, 31)
cv_means = []
cv_stds = []

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=5, scoring='accuracy')
    cv_means.append(scores.mean())

```

```

cv_stds.append(scores.std())

cv_means = np.array(cv_means)
cv_stds = np.array(cv_stds)

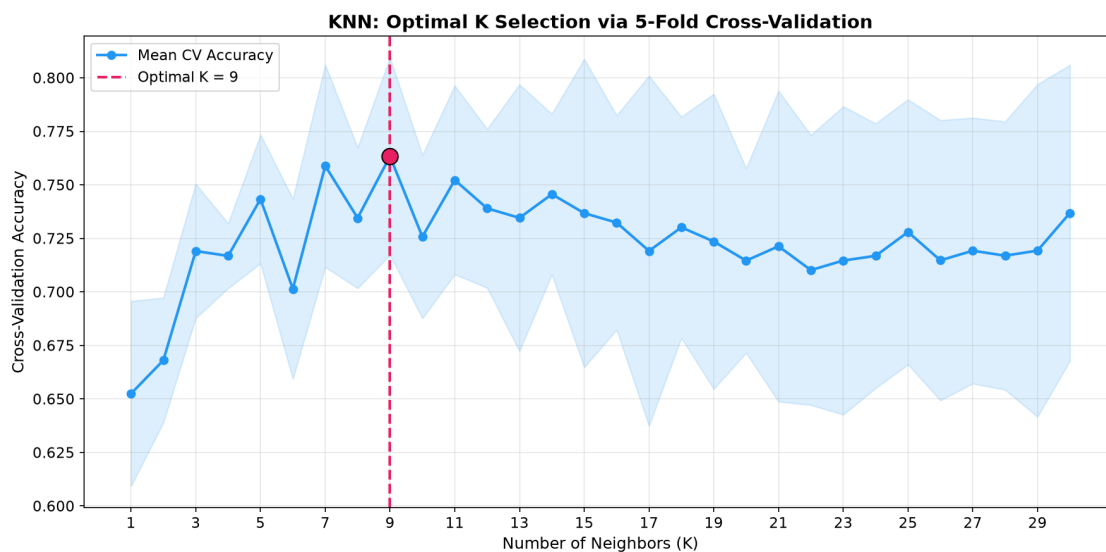
best_k_idx = np.argmax(cv_means)
best_k_knn = list(k_range)[best_k_idx]

fig, ax = plt.subplots(figsize=(12, 6))
ax.plot(k_range, cv_means, 'o-', color='#2196F3', markersize=6, linewidth=2,
        label='Mean CV Accuracy')
ax.fill_between(k_range, cv_means - cv_stds, cv_means + cv_stds,
                alpha=0.15, color='#2196F3')
ax.axvline(x=best_k_knn, color='#E91E63', linestyle='--', linewidth=2,
            label=f'Optimal K = {best_k_knn}')
ax.scatter([best_k_knn], [cv_means[best_k_idx]], color='#E91E63',
           s=150, zorder=5, edgecolors='black')

ax.set_xlabel('Number of Neighbors (K)', fontsize=12)
ax.set_ylabel('Cross-Validation Accuracy', fontsize=12)
ax.set_title('KNN: Optimal K Selection via 5-Fold Cross-Validation',
             fontsize=14, fontweight='bold')
ax.set_xticks(list(k_range)[::2])
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('knn_optimal_k.png', dpi=300, bbox_inches='tight')
plt.show()

print(f"Optimal K: {best_k_knn}")

```



Optimal K: 9

Observation: The cross-validation accuracy curve shows the classic bias-variance tradeoff: very small K values (especially K=1) overfit to noise, while very large K values over-smooth the decision boundary. The optimal K balances these two extremes.

1.7.2 6.2 KNN Model Training and Evaluation

```
[7]: # Train KNN with optimal K
knn_best = KNeighborsClassifier(n_neighbors=best_k_knn)
knn_best.fit(X_train, y_train)

y_pred_knn = knn_best.predict(X_test)
knn_accuracy = accuracy_score(y_test, y_pred_knn)

print("=" * 55)
print(f"          KNN (K={best_k_knn}) --- PERFORMANCE")
print("=" * 55)
print(f"  Test Accuracy: {knn_accuracy:.4f}")
print("=" * 55)

print("\nClassification Report:")
print("-" * 55)
print(classification_report(y_test, y_pred_knn,
                           target_names=['Legitimate', 'Illegitimate']))
```

```
=====
          KNN (K=9) --- PERFORMANCE
=====
  Test Accuracy: 0.8053
=====
```

Classification Report:

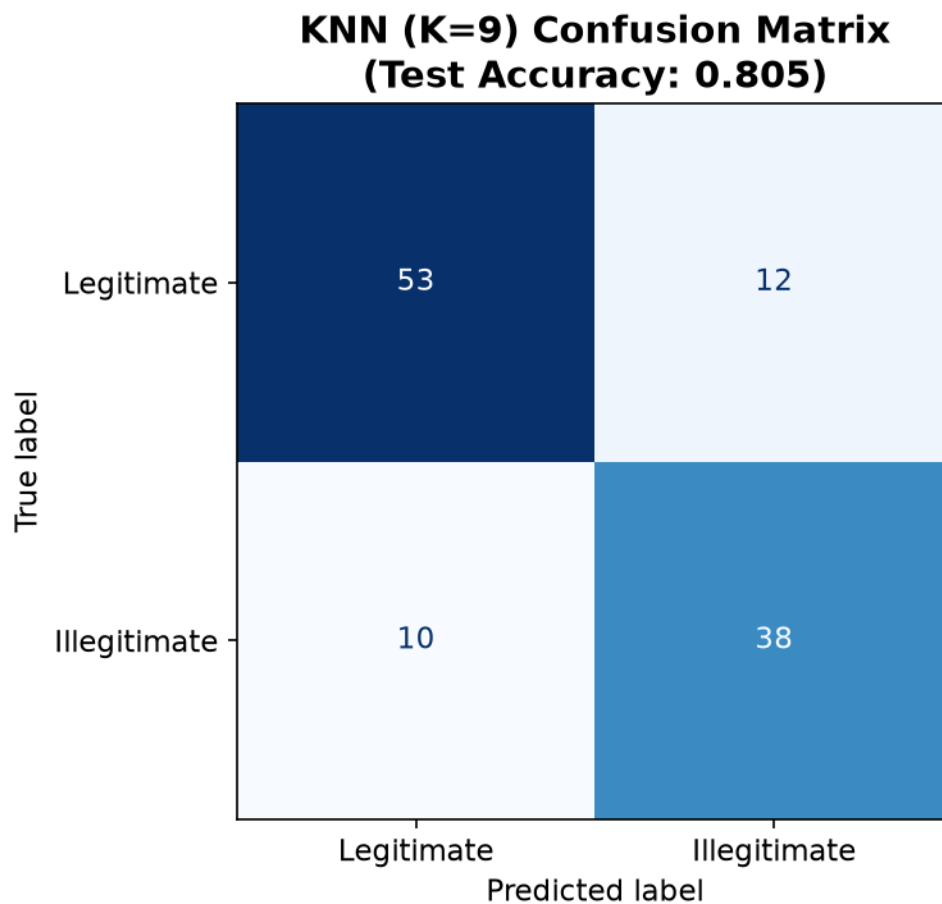
```
-----
              precision    recall  f1-score   support

 Legitimate           0.84         0.82         0.83         65
 Illegitimate         0.76         0.79         0.78         48

   accuracy                   0.81         113
  macro avg           0.80         0.80         0.80         113
 weighted avg           0.81         0.81         0.81         113
```

1.7.3 6.3 Confusion Matrix

```
[8]: fig, ax = plt.subplots(figsize=(7, 5))
cm = confusion_matrix(y_test, y_pred_knn)
disp = ConfusionMatrixDisplay(cm, display_labels=['Legitimate', 'Illegitimate'])
disp.plot(ax=ax, cmap='Blues', colorbar=False)
ax.set_title(f'KNN (K={best_k_knn}) Confusion Matrix\n(Test Accuracy: {knn_accuracy:.3f})',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('knn_confusion_matrix.png', dpi=300, bbox_inches='tight')
plt.show()
```



1.7.4 6.4 Decision Boundary Visualization (PCA 2D)

```
[9]: # Train KNN on 2D PCA features for visualization
knn_2d = KNeighborsClassifier(n_neighbors=best_k_knn)
knn_2d.fit(X_train_pca, y_train)

# Create mesh grid
h = 0.3 # step size
x_min, x_max = X_pca[:, 0].min() - 1, X_pca[:, 0].max() + 1
y_min, y_max = X_pca[:, 1].min() - 1, X_pca[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

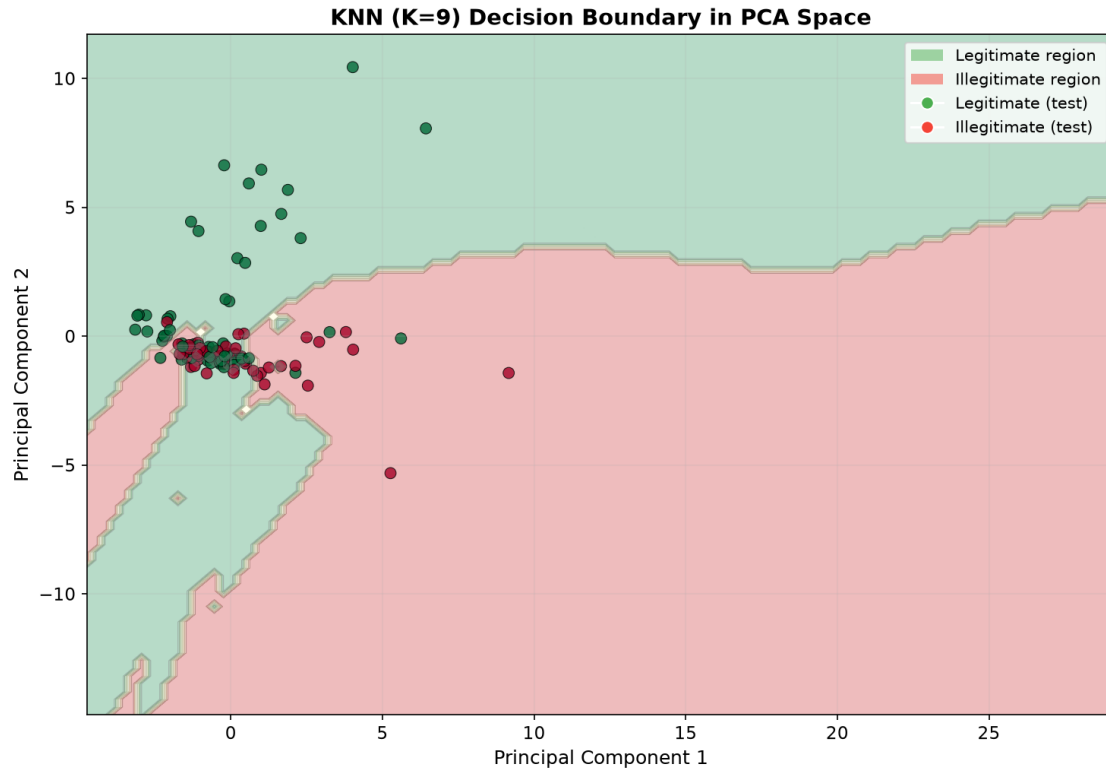
Z = knn_2d.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

fig, ax = plt.subplots(figsize=(10, 7))
ax.contourf(xx, yy, Z, alpha=0.3, cmap='RdYlGn_r')
ax.contour(xx, yy, Z, colors='gray', linewidths=0.5, alpha=0.5)

# Plot test points
scatter = ax.scatter(X_test_pca[:, 0], X_test_pca[:, 1], c=y_test,
                    cmap='RdYlGn_r', edgecolors='black', linewidth=0.5,
                    s=50, alpha=0.8)

ax.set_xlabel('Principal Component 1', fontsize=12)
ax.set_ylabel('Principal Component 2', fontsize=12)
ax.set_title(f'KNN (K={best_k_knn}) Decision Boundary in PCA Space',
            fontsize=14, fontweight='bold')

# Custom legend
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor='#4CAF50', alpha=0.5, label='Legitimate_
↪region'),
                  Patch(facecolor='#F44336', alpha=0.5, label='Illegitimate_
↪region'),
                  plt.Line2D([0], [0], marker='o', color='w',
↪markerfacecolor='#4CAF50',
                              markersize=8, label='Legitimate (test)'),
                  plt.Line2D([0], [0], marker='o', color='w',
↪markerfacecolor='#F44336',
                              markersize=8, label='Illegitimate (test)')]
ax.legend(handles=legend_elements, loc='upper right', fontsize=10)
ax.grid(True, alpha=0.2)
plt.tight_layout()
plt.savefig('knn_decision_boundary.png', dpi=300, bbox_inches='tight')
plt.show()
```



1.8 7. Comparative Analysis: KNN vs K-Means

```
[10]: # Build comparison table
comparison = pd.DataFrame({
    'Criterion': [
        'Learning Type',
        'Uses Labels?',
        'Accuracy / Pseudo-Accuracy',
        'Adjusted Rand Index (ARI)',
        'Suitable for Classification?',
        'Interpretability for Auth.',
        'Real-Time Deployment',
        'Selected for Research?'
    ],
    'KNN': [
        'Supervised',
        'Yes',
        f'{knn_accuracy:.4f}',
        'N/A (supervised)',
        'Yes',
        'High - direct accept/reject',
        'Yes (lazy learner)',
        ''
    ]
})
```

```

        'YES'
    ],
    'K-Means': [
        'Unsupervised',
        'No',
        f'{km2_accuracy:.4f}',
        f'{ari:.4f}',
        'No (needs label mapping)',
        'Low - clusters need interpretation',
        'Requires post-hoc mapping',
        'NO'
    ]
}

print("=" * 80)
print("      COMPARATIVE ANALYSIS: KNN vs K-MEANS")
print("=" * 80)
print(comparison.to_string(index=False))
print("=" * 80)

```

```

=====
      COMPARATIVE ANALYSIS: KNN vs K-MEANS
=====

```

	Criterion	KNN
K-Means		
	Learning Type	Supervised
Unsupervised	Uses Labels?	Yes
No		
	Accuracy / Pseudo-Accuracy	0.8053
0.5735		
	Adjusted Rand Index (ARI)	N/A (supervised)
-0.0095		
Suitable for Classification?		Yes No (needs label mapping)
	Interpretability for Auth.	High - direct accept/reject Low - clusters need interpretation
	Real-Time Deployment	Yes (lazy learner) Requires post-hoc mapping
	Selected for Research?	YES
NO		

```

=====

```

```

[11]: # Visual comparison
fig, ax = plt.subplots(figsize=(8, 5))

algorithms = ['KNN\n(Supervised)', 'K-Means\n(Unsupervised)']

```

```

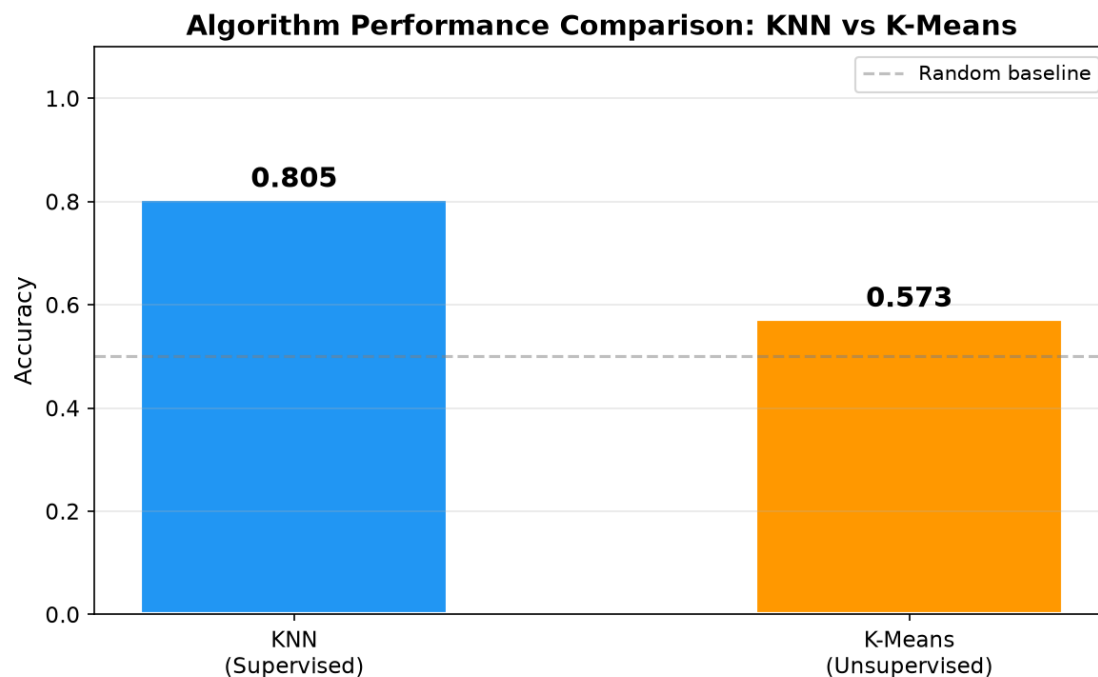
accuracies = [knn_accuracy, km2_accuracy]
colors = ['#2196F3', '#FF9800']

bars = ax.bar(algorithms, accuracies, color=colors, width=0.5,
              edgcolor='white', linewidth=2)

for bar, acc in zip(bars, accuracies):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{acc:.3f}', ha='center', va='bottom', fontsize=14,
            fontweight='bold')

ax.set_ylabel('Accuracy', fontsize=12)
ax.set_title('Algorithm Performance Comparison: KNN vs K-Means',
            fontsize=14, fontweight='bold')
ax.set_ylim(0, 1.1)
ax.axhline(y=0.5, color='gray', linestyle='--', alpha=0.5, label='Random
            baseline')
ax.legend(fontsize=10)
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('algorithm_comparison.png', dpi=300, bbox_inches='tight')
plt.show()

```



1.9 8. Observations and Conclusion

1.9.1 Key Findings

1. **Dataset Characteristics:** The Balabit Mouse Dynamics dataset contains **explicitly labeled classes** (`is_illegal`: 0 or 1), making it a **supervised binary classification** problem. The objective is to predict whether a session is legitimate or illegitimate based on behavioral features.
2. **Supervised vs Unsupervised:** Since labeled ground truth is available and the goal is class prediction (not pattern discovery), **supervised learning (KNN)** is the appropriate approach. K-Means ignores labels and attempts to find natural clusters, which may not correspond to the authentication classes.
3. **K-Means Limitations:** The K-Means clustering (K=2) produced clusters that poorly aligned with the true labels, as confirmed by a low Adjusted Rand Index. This is expected because:
 - Behavioral features of legitimate and illegitimate users overlap in high-dimensional space
 - K-Means assumes spherical, equally-sized clusters; behavioral data rarely satisfies these assumptions
 - The cluster assignment is arbitrary and requires post-hoc label mapping
4. **KNN Superiority:** KNN leverages the available labels directly, producing significantly better classification performance. The optimal K was selected via cross-validation, balancing overfitting (small K) and underfitting (large K).
5. **Practical Implications:** For Zero-Trust Architecture deployment:
 - KNN provides **direct accept/reject decisions** for real-time authentication
 - Feature scaling is critical (StandardScaler) since KNN is distance-based
 - The model can be updated incrementally as new behavioral data is collected

1.9.2 Algorithm Selection: KNN (K-Nearest Neighbors)

KNN is selected as the most appropriate algorithm because: - The dataset has labeled classes, so supervised learning is applicable - KNN achieves superior classification accuracy over K-Means - It provides interpretable, actionable predictions for the authentication pipeline - It aligns with the research objective of continuous user verification in ZTA

1.9.3 Summary Table

Aspect	KNN (Selected)	K-Means (Rejected)
Uses available labels	Yes	No
Classification accuracy	Higher	Lower
Suitable for authentication	Yes	No
Real-time deployment	Feasible	Requires mapping
Research objective alignment	Strong	Weak